

HANDOUT TO THE HPP-CA PRESENTATION

Backroung

xxxKINESW
High-Bit Low-Bit

HPP is an Cellular-Automaton (CA) to simmlate gas-particle movement commonly used for cryptograpy because of its symmetrical properties.

We chose an 2D-grid with an van Neumann neighbourhood and an toroidal shape.

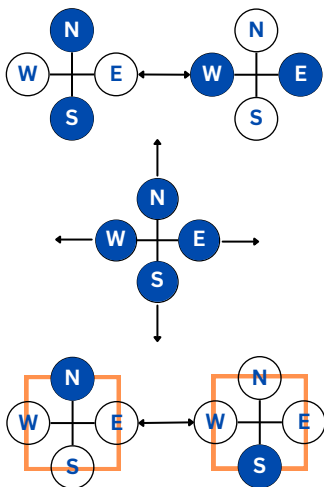
Each cell is represt by one byte. Where only the Low-Bits are translated into North (N), East(E), South (S) and West(W) moving particles.

The **key/ wall** bit is the first High-Bit and determns wether there is an wall which changes the applied rule.

The rules are applied to the cells sequentially. The order depends on wether one wants to **encrypt** (forward) or **decrypt** (backward).

Forward: Collision → Propagation → Reflection → Collision → ...

Backward: Refleciton → Inverse-Propagation → Collision → Reflection → ...



Collision

Two particles that meet head-on are rotated by 90 degrees

Propagation

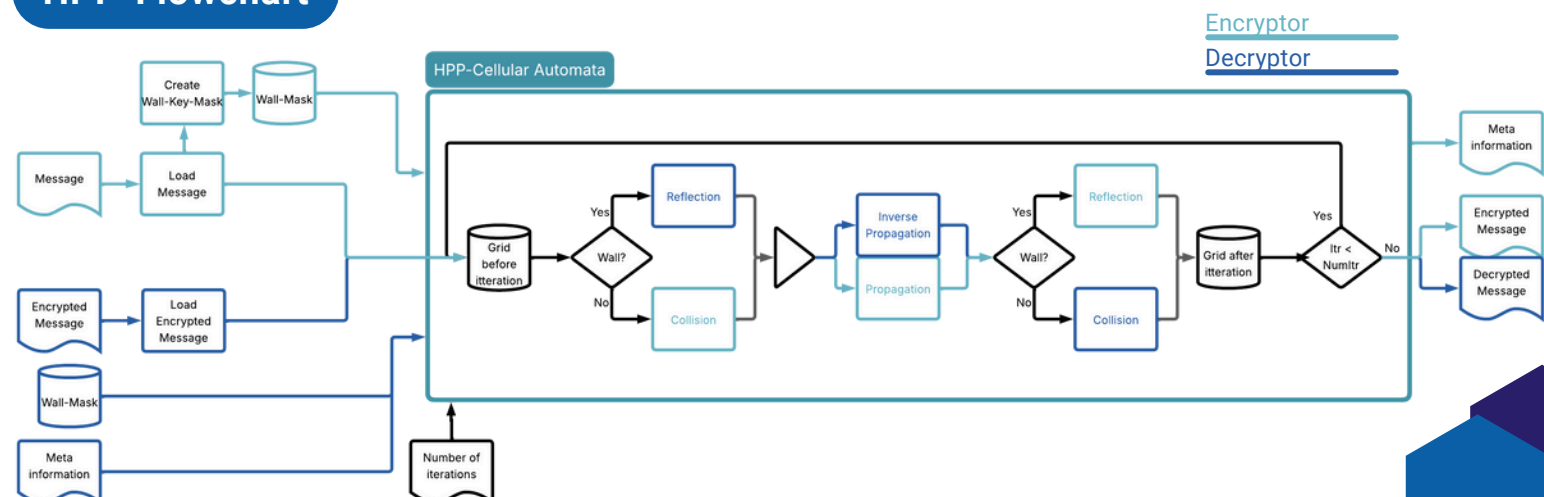
Each particle can propagate to its corresponding adjacent cell. When running the CA backwards the particles propagate in the opposite direction

Reflection

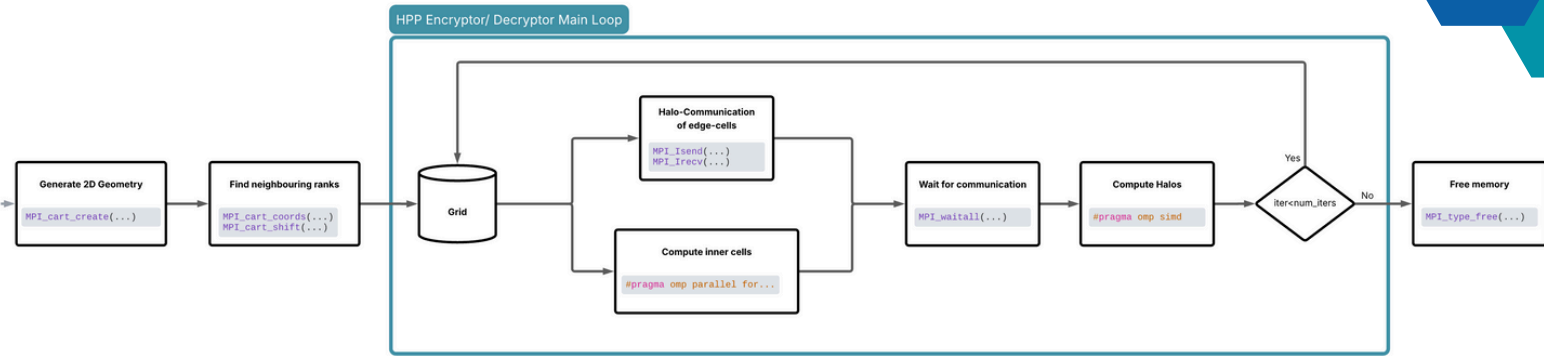
When there is a wall in the cell a particle is rotated by 180 degrees

HPP-Rules

HPP-Flowchart



Parallalization Structur



- Heavy lifting happens in main-loop → focus on that
- Use domain-decompositon for MPI parallelization → every process its domain
- Define domaines so that volume/edge ratio is optimized
- Transfer information between domains via Halo cells
- Use non-blocking communication → inner core can be computed whilst exchanging Halos
- Use downstream MPI_Waitall for synchronization
- After that: Can savely compute border cells
- Use “#pragma omp parallel for” when looping over inner cells
- May want to use double-buffer system to calculate next grid state

Essentials:

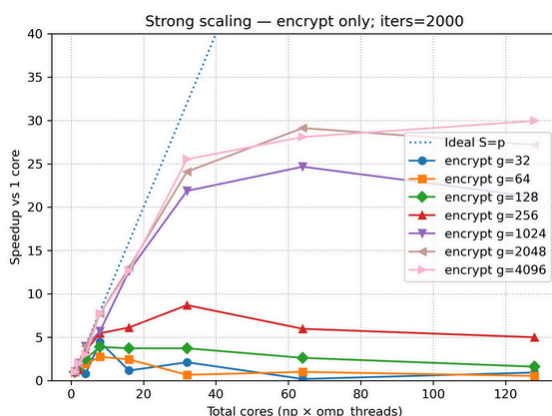
MPI_Dims, MPI_Cart family (geometry, coordinates, neighbor identification)

MPI_Isend, MPI_Irecv, MPI_Waitall (non-blocking Halo exchange, synchronization)

#pragma omp parallel for, #pragma omp simd (loop speedups)

Results

Strong and Weak scaling behaviour



An ideal speed-up would follow the blue dotted line.

Our algorithm shows an ideal speed up for:

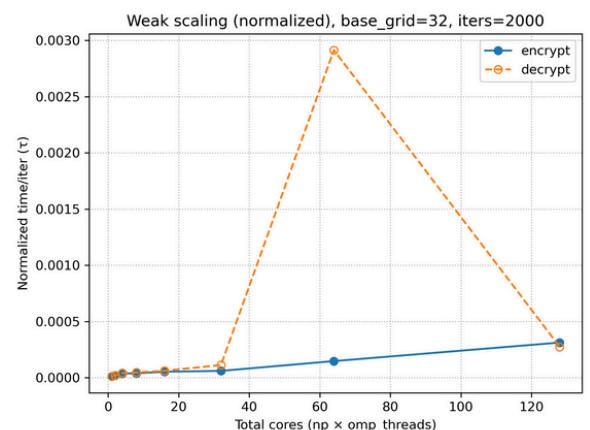
- large grids (2^{12}) up to 32 total cores used
- medium grids (2^{10}) up to 16 total cores
- small grids (2^5) up to 8 cores

One reason for this behaviour is that the communication overheads for a certain amount of total cores is greater than the gain in computational power.

A perfect weak-scaling would be an horizontal line meaning the normalized time remains constant regardless of the number of cores used.

Both the decryption and encryption shows not ideal weak scaling suggesting the communication between processes/ threads takes additional time. We treat the one (64 toatl cores) as an Outlier with no significant meaning.

Yet the time “lost” has a magnitude of 10^{-4} seconds. This can be interpreted as not ideal bur quiet good behaviour for weak scaling.



[1] Signac, Laurent. (2013). Lattice Gas Symmetric Cryptography.

[2] Molekulardynamik und Lattice-Boltzmann Methoden – Dr. Matrin Bernreuther, Universität Stuttgart, SS 2005

[3] Wikipedia: Cellular automaton